

RepoTransBench: A Real-World Multilingual Benchmark for Repository-Level Code Translation

Yanli Wang¹, Yanlin Wang², Suiquan Wang³, Daya Guo⁴, Jiachi Chen⁵, *Member, IEEE*, John Grundy⁶, *Fellow, IEEE*, Xilin Liu⁷, Yuchi Ma⁸, Mingzhi Mao⁹, Hongyu Zhang¹⁰, and Zibin Zheng¹¹, *Fellow, IEEE*

Abstract—Repository-level code translation refers to translating an entire code repository from one programming language to another while preserving the functionality of the source repository. Many benchmarks have been proposed to evaluate the performance of such code translators. However, previous benchmarks mostly provide fine-grained samples, focusing at either code snippet, function, or file-level code translation. Such benchmarks do not accurately reflect real-world demands, where entire repositories often need to be translated, involving longer code length and more complex functionalities. To address this gap, we propose a new benchmark, named RepoTransBench, which is a real-world multilingual repository-level code translation benchmark featuring 1,897 real-world repository samples across 13 language pairs with automatically executable test suites. Besides, we introduce RepoTransAgent, a general agent framework to perform repository-level code translation. We evaluate both our benchmark's challenges and agent's effectiveness using several methods and backbone LLMs, revealing that repository-level translation remains challenging, where the best-performing method achieves only a 32.8% success rate. Furthermore, our analysis reveals that translation difficulty varies significantly by language pair direction, with dynamic-to-static language translation being much more challenging than the reverse direction (achieving below 10% vs. static-to-dynamic at 45-63%). Finally, we conduct a detailed error analysis and highlight current LLMs' deficiencies in repository-level code translation, which could provide a reference for further improvements. We provide the code and data at <https://github.com/DeepSoftwareAnalytics/RepoTransBench>.

Index Terms—Repository-level code translation, multilingual benchmark, LLM-based agent.

I. INTRODUCTION

CODE translation refers to translating code from one programming language to another while preserving the functionality of the source code [1], [2], [3], [4]. Recently, large language models (LLMs) have demonstrated strong performance across various tasks. However, the training of these models increasingly faces limitations regarding the availability of open-source code data. High-quality synthetic data has become essential for further improving LLMs' performance [5], [6]. By leveraging code translation, vast amounts of code written in various programming languages can be transformed into high-quality training data, offering a viable solution to this scarcity [7], [8], [9]. Beyond data synthesis, code translation has broad applications in other areas. It facilitates the refactoring of code written in outdated languages [10], transitions from simpler but slower languages to more complex and faster ones [11], and supports programming language migration in software development [12], [13], [14], [15], [16], [17], [18], [19]. These capabilities highlight the significance of code translation in addressing diverse challenges across the software engineering domain. Automatic code translation can significantly reduce manual effort and has thus garnered widespread attention in recent years [20], [21], [22], [23], [24], [25], [26], [27], [28]. With the popularity of large language models (LLMs), researchers are trying to translate code with LLMs, yielding promising results [29], [30], [31].

To evaluate the performance of code translation tools, various benchmarks have been introduced [32], [33], [34], [35], [36], [37], [38], [39], [40]. Based on translation granularity, current fine-grained code translation benchmarks can be classified into three levels [1]: snippet-level, function-level, and file-level. Specifically, **snippet-level** code translation benchmarks, such as CoST [32], XLCost [33], typically focus on evaluating the translation of program segments located between two consecutive code comments. **Function-level** code translation benchmarks, such as TransCoder-test [34], CodeXGLUE [35] and HumanEval-X [36], focus on evaluating the translation of a function. **File-level** code translation benchmarks which include CodeNet [37], Avatar [38], CodeScope [39] and CodeTransOcean [40] refer to evaluating the translation of a complete program file. However, these fine-grained code translation benchmarks may not meet the demands of real development scenarios, which require the translation of entire repositories. Recently, Pan et al. [1] manually study two open-source

Received 11 January 2025; revised 29 July 2025 and 29 November 2025; accepted 11 December 2025. Date of publication 17 December 2025; date of current version 13 February 2026. This work was supported by CCF-Huawei Populus Grove Fund CCF-HuaweiSE202403. Recommended for acceptance by M. Kechagia. (Corresponding author: Yanlin Wang.)

Yanli Wang, Yanlin Wang, Suiquan Wang, Daya Guo, Jiachi Chen, Mingzhi Mao, and Zibin Zheng are with Sun Yat-sen University, Guangzhou 510006, China (e-mail: wangyli58@mail2.sysu.edu.cn; wangylin36@mail.sysu.edu.cn).

John Grundy is with Monash University, Melbourne, VIC 3800, Australia. Xilin Liu and Yuchi Ma is with Huawei Cloud Computing Technologies Company, Ltd., Shenzhen 518129, China.

Hongyu Zhang is with Chongqing University, Chongqing 400044, China. Digital Object Identifier 10.1109/TSE.2025.3645056

repositories (Apache Commons CLI [41] and Python Click [42]), and find that current LLMs struggle to perform code translation for entire repositories. Although this work has conducted preliminary research on repository-level code translation, we identify the following problems:

- **P1: Lack of Large-Scale Multilingual Benchmark.** Existing repository-level translation studies are severely limited in scope, with most work examining only one or two translation pairs and a handful of repositories. This narrow coverage fails to capture the diverse challenges posed by different programming paradigms, syntax variations, and ecosystem differences that occur across the broader landscape of programming languages used in real-world development.
- **P2: Labor-Intensive Execution-Based Test Suites Construction.** Repository-level code translation benchmark construction is labor-intensive, requiring extensive manual effort to ensure repository executability, generate comprehensive test suites, validate functional correctness, and manage complex dependencies.
- **P3: Lack of General Translation Framework for Different Translation Pairs.** Current translation methods often rely on language-specific heuristics tailored to particular translation pairs, making them difficult to generalize across diverse programming language combinations.
- **P4: Potentially Ignoring the Meta Information of Repositories.** Many programming language repositories contain configuration files and resource files, such as `CMakeLists.txt` for C++ and `pom.xml` for Java. A real-world code repository migration often requires proper handling of resources and correct configuration management.

In this paper, we introduce a *multilingual repository-level code translation benchmark*, named **RepoTransBench**, and a general *agent-based translation framework*, named **RepoTransAgent**, to address these limitations.

RepoTransBench encompasses 1,897 repository samples across 13 translation pairs covering 7 programming languages, providing automatic execution-based test suites to evaluate both **compilability and functional correctness** of translated repositories. To construct RepoTransBench, we develop a multi-agent framework that automatically generates comprehensive test suites and handles the complex requirements of repository-level translation validation. Our benchmark demonstrates higher complexity than previous work, with repositories containing an average of 23,966 tokens, 2,394 lines of code, 177 functions, 35 classes, and 163 import statements.

RepoTransAgent addresses general translation challenges through an intelligent agent framework based on the Re-Act (Reasoning + Acting) paradigm, specifically designed to solve repository-level code translation problems. The agent iteratively combines reasoning about repository structure and translation requirements with concrete actions to handle the complexity of entire software projects. RepoTransAgent can analyze repository structures, understand cross-file dependencies, manage configuration files, and iteratively refine translations based on execution feedback. The agent operates through five

core capabilities: reading files, creating files, executing commands, searching content, and marking completion, enabling repository-level translation through a reasoning-action loop that adapts to diverse programming language ecosystems.

We evaluate both the benchmark's challenges and our agent's effectiveness using several methods and backbone LLMs. Our experimental results reveal that repository-level code translation remains challenging for current methods, with the best-performing method achieving only 32.8% success rate. However, our RepoTransAgent framework consistently outperforms baseline approaches, demonstrating improvements of up to 21.5% over the error feedback method.

The key contributions of this research are:

- We introduce a large-scale repository-level code translation benchmark named **RepoTransBench** covering 13 translation pairs with 1,897 samples and automatic execution-based test suites. RepoTransBench demonstrates substantially higher context and dependency complexity than previous benchmarks.
- We develop a multi-agent framework for automated benchmark construction to handle the complex requirements of repository-level translation validation and obtain the corresponding execution-based test suites.
- We propose **RepoTransAgent**, a general agent framework for multilingual repository-level code translation based on reasoning and action paradigms, achieving up to 32.8% success rate on RepoTransBench.
- We conduct an extensive evaluation across multiple dimensions, revealing that translation difficulty varies by different translation pairs and project complexity.

II. BACKGROUND

Code translation involves converting source code written in one programming language into another language while preserving the original program's functionality and logic [1], [2], [3], [4]. This process is essential in software engineering for several reasons, such as migrating legacy systems to modern languages [12], [13], improving code performance by translating to more efficient languages [10] and enabling cross-platform compatibility [11]. As programming languages continue to evolve, the demand for accurate and efficient code translation techniques has grown, making it a critical area of research and development [20], [21], [22], [23], [24], [25], [26], [27], [28]. The field has experienced significant technological evolution, with each advancement enabling translation capabilities at increasingly larger code granularities. The development of code translation can be broadly divided into three main approaches based on their underlying technologies: Rule-Based Translation, Neural Network-Based Translation, and Large Language Model-Based Translation.

Rule-Based Translation. The earliest code translation approaches relied on manually defined rules and grammar specifications. Existing non-learning-based code translation techniques can be categorized into several main groups. Parser-based tools like ANTLR [43] rely on manually defined grammar rules to translate source code between languages. Transpilers

such as Babel [44], Emscripten [45], JSweet [46], and GWT [47] convert source code from one language to another, often used to ensure compatibility across platforms or systems. Domain-specific translators like CxGo [48], C2Rust [49], and JavaToCSharp [50] focus on specific translation pairs, offering targeted translation solutions. Intermediate language compilers like Haxe [51] compile code to a variety of target languages by using an intermediate format. Interface generators such as SWIG [52] create cross-language bindings, allowing different languages to interact with each other without direct code translation. While these rule-based approaches demonstrate high precision for well-defined translation patterns, they face significant limitations in handling complex code structures and require substantial manual effort to maintain and extend. Most rule-based methods primarily support **snippet-level** code translation [32], [33], which typically refers to evaluating the translation of program segments that are located between two consecutive code comments, and each program may consist of one or more code snippets. Some advanced rule-based tools can handle simple **function-level** translation where functions can be processed independently without complex external dependencies.

Neural Network-Based Translation. The introduction of neural networks marks a paradigm shift in code translation, enabling more sophisticated translation capabilities through learned representations. Early learning-based approaches often train a neural network to achieve the ability of code translation [24], [25], [27], [53]. Aggarwal et al. [10] convert Python 2 code to Python 3 code using trained Moses [54], which is an open-source toolkit for statistical machine translation. Chen et al. [21] design a tree-to-tree neural network to translate a source tree into a target one. DeepAM [19] discusses the limitations of bilingual projects, as well as the automatic mining of API mappings to reduce manual effort in code migration. Zheng et al. [55] propose an approach for zero-resource NMT using maximum expected likelihood estimation. TransCoder [34] is a transformer with 6 layers to perform code translation at function level. Besides, some models pre-trained on multilingual corpora like Codex [56], CodeT5 [53], and CodeGen [6] demonstrate remarkable code translation capability. These neural approaches successfully enable practical **function-level** translation [32], [33], [34], [35], [36], which refers to translating a function into another programming language, with the data sources often being manually crafted datasets [56] or coding practice websites [57]. More advanced neural systems achieve **file-level** code translation [37], [38], [39], [40], [58], [59], which often refers to translating a complete program file into the target language. The data sources are usually from code contest platforms [60], [61], [62], [63] or task solutions websites [64], [65], [66]. G-TransEval [67] also provides a more fine-grained taxonomy, including token-level, syntax-level, library-level, and algorithm-level, which is part of a function. However, neural network-based approaches still struggle with cross-file dependencies and large-scale context management, limiting their effectiveness beyond file-level translation.

Large Language Model-Based Translation. In recent years, large language models such as StarCoder [68], [69],

SantaCoder [70] and more latest models such as the Llama series [71], [72], [73], the ChatGPT series [74], the DeepSeek series [75], and the Claude series [76] have shown remarkable performance on traditional code translation tasks. These models are trained on large code corpus and have strong comprehension and instruction following abilities, which can perform accurate and efficient code translations on previous fine-grained code benchmarks. Recent research has explored various techniques to enhance LLM-based translation capabilities. Rectifier [59] is a fine-tuned micro model that acts as a general corrector to correct the translation errors of unknown LLMs. Vert [77] leverages LLM's strong few-shot learning ability to produce readable Rust translations with formal guarantees of correctness. Bhattarai et al. [78] enhance code translation in LLMs with few-shot learning via retrieval-augmented generation. TransAgent [79] is an LLM-based multi-agent system for code translation. SpecTra [31] considers the different kinds of specifications that can be extracted from a program to enhance the code translation ability of LLMs. Momoko et al. [80] propose an LLM-based translation scheme that improves the success rate of translating large-scale C code into compilable Rust code. SolMover [81] can convert smart contracts written in Solidity [82] to Move [83] with LLMs. CCTrans [84] can transpile concurrent Java files to JavaScript using multiple workers while maintaining identical behavior. GlueTest [85] systematically and semiautomatically validates translations for non-trivial libraries. UniTrans [29] is a unified code translation framework applicable to various LLMs to unleash their power in code translation. SDA-Trans [86] is a syntax and domain-aware model for program translation, which leverages the syntax structure and domain knowledge to enhance the cross-lingual transfer ability.

Fig. 1 shows a comparison of code granularities used in different code translation approaches across these technological eras. Most previous works focus on the translations with a granularity not exceeding a single code file (left-hand side). Unlike the previous fine-grained granularity code translation, **repository-level** code translation involves migrating an entire repository from one language to another. Recently, repository-level code translation has gradually gained the attention of researchers [1]. As shown in the right-hand side of Fig. 1, a typical code repository contains functional code files and test code files and may also include resource files and configuration files. Functional code files refer to those code files that implement specific functionalities of the code repository, such as the files located in the `readtime/` directory. Test code files refer to the files used to verify the correctness of the functional code where `test_readtime.py` is an example. Resource files like those in `samples/` folder are used to test the functional correctness of the functional code. The functional code needs to implement how to perform I/O operations with these resources. In addition, for certain language-specific frameworks, it is necessary to complete the configuration file correctly, such as the “pom.xml” file in Java's Maven [87] repositories.

Compared with previous fine-grained code translation granularity, repository-level code translation presents three fundamentally different challenges: challenging context management, complex dependency analysis, and difficult environment

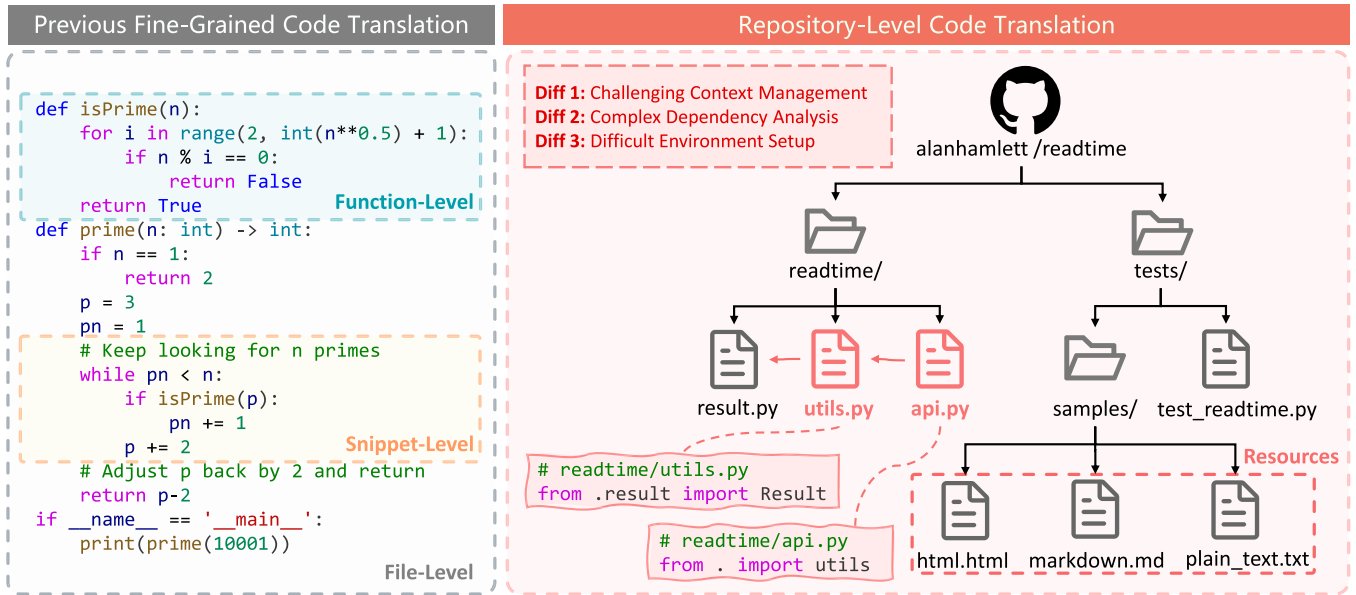


Fig. 1. Comparison of different code translation granularity.

setup. Real-world code repositories typically include numerous functions, classes, and import statements to realize complex functionalities, requiring translators to understand the entire repository context rather than isolated code fragments with intricate interdependencies between components across multiple files and modules. The complexity extends beyond code volume to sophisticated dependency management requirements, where files like `api.py` and `utils.py` have complex import relationships (e.g., `from .result import Result`, `from . import utils`) that must be correctly analyzed and maintained during translation. Additionally, successful repository translation necessitates appropriate configuration of ecosystem-specific files and comprehensive resource migration beyond source code, where resource files that perform I/O operations with code components must be carefully handled and potentially transformed to maintain functional equivalence. Furthermore, unlike artificially crafted datasets used in fine-grained translation [37], [38], real repositories typically contain existing test suites that can serve as valuable validation mechanisms. Effective repository-level translation must leverage these test cases not only for validation but also as specifications for maintaining functional correctness throughout the translation process.

Recent studies have shown interest in repository-level code translation. Pan et al. [1] attempt to perform mutual conversion between Python and Java projects [41], [42], but find that the advanced LLMs are largely ineffective, with success rates of 8.1% for GPT-4 and 0% for the rest of the models. This stark performance gap reveals that repository-level translation requires fundamentally different approaches from fine-grained translation, as current LLM-based methods struggle with the challenges of large-scale context management, complex dependency analysis, and comprehensive environment configuration that are inherent to repository-level code translation.

III. REPOTRANSBENCH

As illustrated in Fig. 2, the data collection pipeline of RepoTransBench consists of three steps: data collection, rule-based filtering, and multi-agent-based test suite construction.

A. Data Collection

We conduct a questionnaire among professional developers to identify practical demands for repository-level code translation. We select the top 7 languages from the TIOBE programming language rankings [88] selecting Rust and Matlab as candidate languages for our translation pair matrix. Additionally, we provide custom options for respondents to specify other translation pairs they require. The detailed questionnaire and statistical results are available in the artifact.

We receive responses from 21 professional developers, comprising 86 requests for specific translation pairs. The survey results reveal diverse translation needs. The highest demand is observed for Python $\leftrightarrow$ C++ bidirectional translation, with developers frequently needing to migrate between Python's rapid prototyping capabilities and C++'s performance-critical applications in systems programming and high-performance computing. JavaScript $\rightarrow$ Python translation also shows strong demand as organizations seek to consolidate their tech stacks by moving web-based logic into Python's rich ecosystem for data processing and machine learning workflows.

The sustained interest in Python $\rightarrow$ Java translation reflects the common enterprise pattern where Python prototypes must be productionized in Java environments for scalability and integration with existing enterprise systems. Meanwhile, C $\rightarrow$ Rust translation demand indicates the growing adoption of Rust for systems programming, where developers aim to modernize legacy C codebases while gaining memory safety guarantees. The interest in Python $\rightarrow$ Go translation similarly reflects

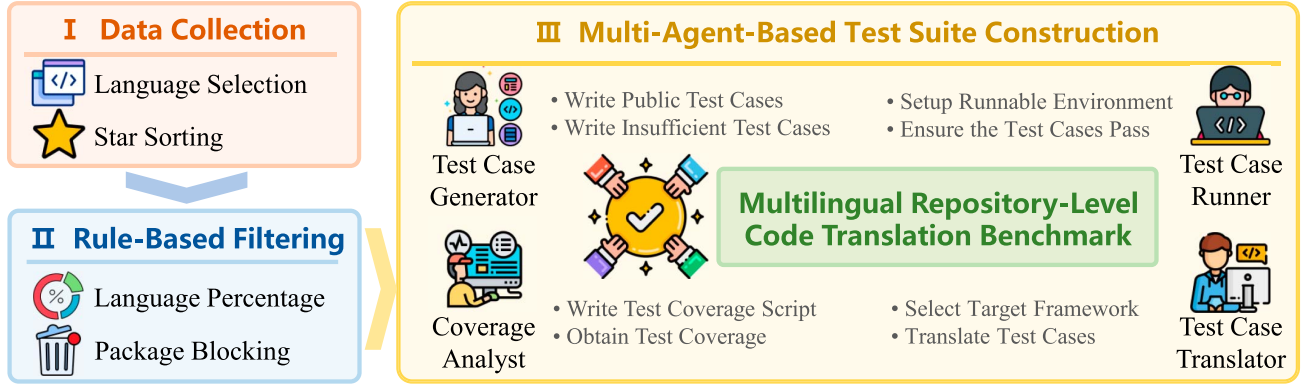


Fig. 2. RepoTransBench construction pipeline.

organizational shifts toward Go’s concurrency model and deployment simplicity for backend services.

Repository-level translation becomes particularly valuable in these scenarios because modern software projects involve complex interdependencies, build systems, and architectural patterns that cannot be addressed through isolated function or class translations. Developers require tools that can maintain semantic correctness across entire codebases while preserving project structure and dependency relationships.

Based on these survey findings, we expand our benchmark to include 13 translation pairs that reflect real-world developer needs. To ensure repository quality, we rank repositories by star count and retain only those with more than 50 stars.

B. Rule-Based Filtering

We provide a specification of our filtering rules in Algorithm 1. The rule-based filtering implements three key criteria: **Rule 1: Language Dominance.** We retain only repositories where the target language constitutes the largest proportion of the codebase. This ensures that the repository is primarily written in the language we aim to translate. **Rule 2: Popularity Threshold.** We filter repositories based on a minimum star threshold θ_{star} to ensure code quality and practical relevance. **Rule 3: Package Exclusion.** We exclude repositories that depend on packages difficult to translate into target languages. The detailed package lists are available at [89]. To illustrate, for Python repositories, we require that (1) Python code comprises the largest share of the codebase, (2) the repository exceeds the popularity threshold, and (3) it does not have substantial dependencies on excluded packages (e.g., PyTorch, TensorFlow, NLTK).

C. Multi-Agent-Based Test Suite Construction

Due to the substantial human resources and effort required for benchmark construction, and to facilitate scalable expansion of dataset size and translation pair types, we develop a multi-agent framework for constructing execution-based repository-level code translation benchmarks. The framework comprises four specialized agents that work collaboratively to ensure

Algorithm 1 Rule-Based Repository Filtering

Input: Collection of repositories $\mathcal{R} = \{R_1, R_2, \dots, R_n\}$, Language L_{target} , Star threshold θ_{star}
Output: Filtered repository collection $\mathcal{R}_{filtered} \subseteq \mathcal{R}$

```

1: function PASSESALLRULES( $R, L_{target}, \theta_{star}$ )
2:   // Rule 1: Language Dominance
3:   if  $P(L_{target}, R) \neq \max\{P(L_i, R) \mid L_i \in \text{Languages}(R)\}$ 
4:     then
5:       return false
6:   end if
7:   // Rule 2: Popularity Threshold
8:   if  $\text{Stars}(R) < \theta_{star}$  then
9:     return false
10:  end if
11:  // Rule 3: Package Blocking
12:  if  $R \cap \text{BlockedPackages}(L_{target}) \neq \emptyset$  then
13:    if not  $\text{IsBasicUsage}(R)$  then
14:      return false
15:    end if
16:  end if
17:  return true
18: end function
19:  $\mathcal{R}_{filtered} \leftarrow \emptyset$ 
20: for each  $R \in \mathcal{R}$  do
21:   if PASSESALLRULES( $R, L_{target}, \theta_{star}$ ) then
22:     $\mathcal{R}_{filtered} \leftarrow \mathcal{R}_{filtered} \cup \{R\}$ 
23:   end if
24: end for
25: return  $\mathcal{R}_{filtered}$ 

```

high-quality benchmark construction: Test Case Generator, Test Case Runner, Coverage Analyst, and Test Case Translator.

Test Case Generator analyzes the source repository structure and functionality to generate comprehensive test cases. It performs two primary functions: (1) writing public test cases that cover the main functionality and API interfaces of the repository, ensuring that critical code paths are exercised, and (2) identifying insufficient test cases by analyzing code coverage gaps and generating additional test scenarios to improve overall test completeness.

Test Case Runner focuses on environment setup and test execution validation. This agent is responsible for: (1) setting up runnable environments by analyzing project dependencies,

TABLE I

STATISTICS OF REPOTRANSBENCH COMPARED TO OTHER EXISTING CODE TRANSLATION DATASETS. CONSIDERING THE TRANSLATION OF PYTHON TO JAVA, WE REPORT THE NUMBER OF SAMPLES AND THE AVERAGE NUMBER OF TOKENS, LINES, FUNCTIONS, CLASSES, AND IMPORT STATEMENTS PER SAMPLE OF EACH TASK. # TOKENS COUNTS ARE BASED ON OPENAI'S TIKTOKEN TOKENIZER (<https://github.com/openai/tiktoken>). # FUNCS, # CLASSES AND # IMPORTS COUNTS ARE BASED ON TREE-SITTER (<https://tree-sitter.github.io>). CODE COMMENTS ARE REMOVED BEFORE COMPUTATION

Dataset	Year	Source	Level	Config File	Evaluation	#Samples	#Tokens	#Lines	#Funcs	#Classes	#Imports
TransCoder-test [34]	2020	GeeksforGeeks [57]	Function	Not Req	Execution	868	127	12	1	0	0
CodeNet [37]	2021	AIZU [62], AtCoder [61]	File	Not Req	Execution	200	99	12	1	0	0
Avatar [38]	2021	AtCoder [61], etc ¹	File	Not Req	Execution	250	175	18	1	0	1
CoST [32]	2022	GeeksforGeeks [57]	Snippet	Not Req	Similarity	351	33	3	0	0	0
			Function	Not Req	Similarity	69	173	15	1	0	0
XLCoST [33]	2022	GeeksforGeeks [57]	Snippet	Not Req	Similarity	6861	24	2	0	0	0
			Function	Not Req	Similarity	864	196	19	1	0	0
HumanEval-X [36]	2022	HumanEval [56]	Function	Not Req	Execution	164	65	8	1	0	0
G-TransEval [67]	2023	HumanEval [56], etc ²	Function ³	Not Req	Similarity	400	90	10	1	0	0
xCodeEval [58]	2023	Codeforces [60]	File	Not Req	Execution	1942	209	22	1	0	1
CodeScope [39]	2023	Codeforces [60]	File	Not Req	Execution	30	259	28	1	0	1
CodeTransOcean [40]	2023	Rosetta Code [66], d2l-ai [65]	File	Not Req	Similarity	1029	253	24	2	0	1
UniTrans [29]	2024	GeeksforGeeks [57]	Function	Not Req	Execution	568	112	11	1	0	0
RepoTransBench	2024	GitHub	Repository	Require	Execution	1897	23966	2394	177	35	163

TABLE II

STATISTICS OF SOURCE LANGUAGES SHOWING CROSS-FILE/INTRA-FILE DEPENDENCIES AND TEST COVERAGE. SRC. LANG.: SOURCE LANGUAGE, PROJ.: PROJECTS, SAMP.: SAMPLES, CROSS.: CROSS-FILE DEPENDENCIES, INTRA.: INTRA-FILE DEPENDENCIES, COV.: COVERAGE, TRANSLATION PAIRS: C→{PYTHON, RUST}, C++→PYTHON, C#→JAVA, JAVA→{C#, GO, PYTHON}, JAVASCRIPT→PYTHON, MATLAB→PYTHON, PYTHON→{C++, GO, JAVA, RUST}. TOTAL: 13 PAIRS, 1897 TRANSLATIONS

Src. Lang.	#Proj.	#Samp	#Cross.	#Intra.	Line Cov.	Branch Cov.
C	122	244	64.3	2494.4	91.71%	63.59%
C++	181	181	94.9	2401.3	86.20%	58.98%
C#	97	97	31.1	600.9	83.52%	80.21%
Java	146	438	156.9	1286.5	73.21%	66.86%
JavaScript	189	189	27.0	707.5	93.56%	88.12%
Matlab	64	64	2.1	1609.5	61.33%	55.99%
Python	171	684	128.1	1006.1	81.29%	79.26%
Overall	970	1897	87.9	1443.7	81.89%	72.61%

installing required packages, and configuring build systems (such as Maven for Java projects or pip for Python projects), and (2) ensuring all test cases pass successfully in the source language environment.

Coverage Analyst provides a quantitative assessment of test quality through comprehensive coverage analysis. This agent: (1) writes language-specific test coverage scripts tailored to each programming language's testing frameworks and coverage tools, and (2) obtains detailed test coverage metrics including line coverage, branch coverage, and function coverage. The coverage analysis ensures that our benchmark maintains high-quality standards, with our current dataset achieving an average of 81.89% line coverage and 72.61% branch coverage across all translation pairs as shown in Table II.

Test Case Translator handles the cross-language translation of test cases to ensure translated repositories can be properly validated. This agent: (1) selects appropriate target frameworks by analyzing the functionality requirements and identifying equivalent libraries and testing frameworks in the target language, and (2) translates test cases from the source language

to the target language while preserving test semantics and assertions. The agent maintains a mapping of equivalent libraries and frameworks across different programming languages to ensure translated test cases accurately reflect the original test intentions.

These agents can perform various operations, including file reading/writing, directory traversal, package installation, search operations, and executing command-line instructions. Through multi-agent collaboration and iterative refinement, the framework operates in a coordinated pipeline where each agent's output serves as input for subsequent agents, ultimately ensuring that source code repositories are executable and successfully generating corresponding test cases in target languages.

D. Statistics of RepoTransBench

Tables I and II present comprehensive statistics of RepoTransBench. Table I compares RepoTransBench with existing code translation benchmarks, while Table II provides detailed statistics across different source languages in our benchmark. Our benchmark represents a paradigm shift from previous fine-grained approaches to repository-level translation, demonstrating unprecedented scale and complexity in code translation evaluation.

Scale and Length Comparison. As shown in Table I, unlike previous benchmarks that operate at function, snippet, or single-file levels, RepoTransBench operates at the repository-level, encompassing complete software projects with their inherent complexity. Our benchmark contains 1,897 translation samples across 13 translation pairs, significantly surpassing the scale of previous work. The average sample in our benchmark contains 23,966 tokens and 2,394 lines of code, representing a dramatic increase of over 95× in tokens and 108× in lines compared to the largest previous benchmark (xCodeEval with 209 tokens and 22 lines per sample).

Structural Complexity. As shown in Table I, the repository-level nature of our benchmark introduces structural elements absent in previous work. Each sample contains an average of

177 functions, 35 classes, and 163 import statements, demonstrating the complex interdependencies and architectural patterns inherent in real-world software projects. In contrast, previous benchmarks typically contain at most 2 functions per sample and rarely include classes or substantial import dependencies.

Dependency Complexity. As shown in Table II, RepoTransBench exhibits substantial dependency complexity that distinguishes it from fine-grained benchmarks. RepoTransBench has an average of 87.9 cross-file dependencies and 1,443.7 intra-file dependencies. The diversity in dependency complexity across different programming languages reflects varied architectural patterns: cross-file dependencies range from 2.1 in Matlab to 156.9 in Java, while intra-file dependencies range from 600.9 in C# to 2,494.4 in C. The complexity distinguishes repository-level translation from function-level or file-level translation, where such cross-module dependencies are absent or minimal.

Configuration Requirements. As shown in Table I, another crucial distinction of RepoTransBench is the requirement for proper configuration files (such as “pom.xml” for Java Maven projects, “package.json” for JavaScript projects, or “requirements.txt” for Python projects). This requirement reflects real-world translation scenarios where successful code migration depends not only on translating source code but also on correctly configuring build systems, dependency management, and project structure in the target language ecosystem.

Cross-Language Coverage. Table II provides detailed statistics across our 13 supported translation pairs, covering translations from C, C++, C#, Java, JavaScript, Matlab, and Python to various target languages. Our dataset encompasses 970 unique projects, with Python being the most represented source language (684 samples from 171 projects) followed by Java (438 samples from 146 projects).

Test Coverage Quality. Our multi-agent framework ensures high-quality test coverage across all translation pairs. As shown in Table II, the overall dataset maintains 81.89% line coverage and 72.61% branch coverage, with JavaScript achieving the highest coverage (93.56% line coverage, 88.12% branch coverage) and Matlab showing the most room for improvement (61.33% line coverage, 55.99% branch coverage). This comprehensive test coverage enables reliable execution-based evaluation of translation quality, moving beyond similarity metrics used in previous benchmarks to assess actual functional correctness.

Translation Pair Classification. We classify the 13 translation pairs from the type system perspective, which can be categorized into four groups: static-to-dynamic translations (C/C++/Java→Python, Matlab→Python), dynamic-to-static translations (Python→C++/Java/Go/Rust), static-to-static translations (C→Rust, C#→Java, Java→C#/Go), and dynamic-to-dynamic translation (JavaScript→Python).

IV. REPOTRANSAGENT

We propose RepoTransAgent, a general agent framework designed for repository-level code translation. RepoTransAgent adopts a holistic understanding of entire software repositories,

enabling it to handle complex interdependencies, maintain project structure, and ensure functional correctness during translation.

A. Agent Architecture and Action Space

As illustrated in Fig. 3, RepoTransAgent operates through an action space consisting of five core capabilities that enable repository analysis and translation:

ReadFile allows the agent to examine existing code files, configuration files, documentation, and dependency specifications. This action is crucial for understanding the repository structure, identifying key components, and analyzing code patterns. The agent uses this capability to verify translation correctness and debug issues during the translation process.

CreateFile enables the agent to generate new files in the target language, including code files, configuration files (such as pom.xml for Java projects or package.json for JavaScript projects), and build scripts. This action is essential for establishing the translated repository structure and ensuring proper project organization in the target language ecosystem.

ExecuteCommand provides the agent with the ability to execute system commands for environment setup, dependency installation, compilation, and testing. This capability enables the agent to validate translations by running build processes and executing test cases, ensuring that the translated repository maintains functional correctness.

SearchContent allows the agent to efficiently locate specific code patterns, function definitions, class declarations, and import statements across the repository. This search capability is particularly important for understanding cross-file dependencies and ensuring consistent translation of related components throughout the project.

Finished marks the completion of the translation task, indicating that the agent has successfully translated the repository and validated its correctness through execution-based testing.

B. Translation Process and Execution Workflow

RepoTransAgent follows a translation workflow based on the ReAct paradigm that begins with comprehensive repository analysis and proceeds through iterative translation and validation cycles. At the start of each translation task, the agent is initialized as a code translator and provided with basic information about the current working directory and translation requirements, including the source language, target language, project name, and other relevant details.

The translation process initiates with the agent’s reasoning phase, where it formulates a plan in the [Thought] section to analyze the repository structure systematically. As demonstrated in the example execution sequence, the agent first reasons that it needs to examine the files in the project directory, then executes the corresponding action using `ExecuteCommand(command="ls -a")` to explore the repository contents. The system provides feedback to the agent through two components: [Result] indicating the execution status (successfully executed in this case), and [Observation] containing the actual command output that reveals source files,

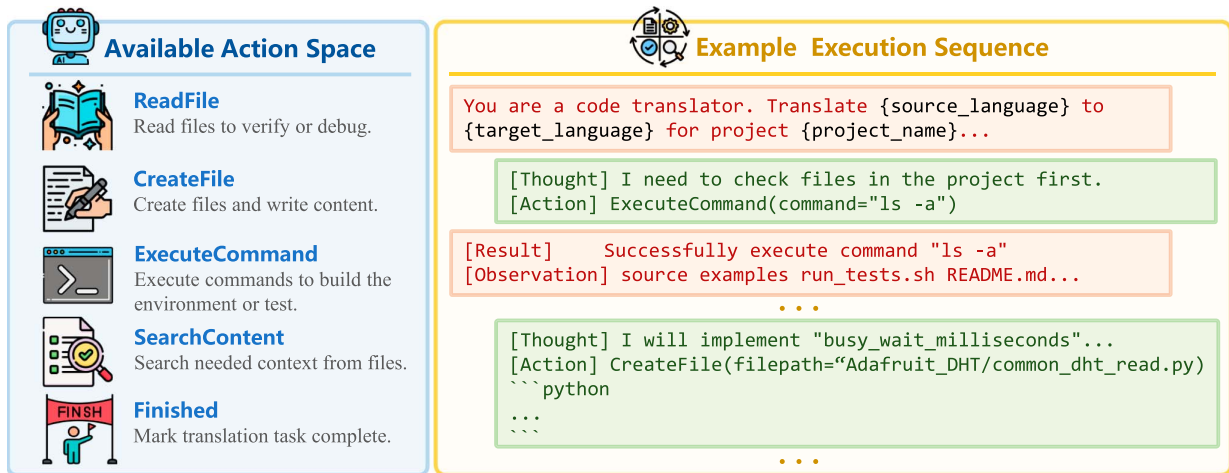


Fig. 3. Overview of the RepoTransAgent.

examples, test scripts, documentation, and other repository components.

Through multiple rounds of repository analysis, the agent develops a comprehensive understanding of the project structure and dependencies. Once sufficient context is gathered, the agent begins the translation process by identifying critical components that require translation, such as core functionality modules. The agent employs a thoughtful approach where it reasons about implementation requirements and develops targeted translation strategies. For example, when encountering a function like `busy_wait_milliseconds`, the agent uses `CreateFile(filepath=". . .")` to create the corresponding Python file and implements the function while preserving the original functionality and adapting it appropriately for the target language ecosystem.

Throughout the entire translation process, the agent maintains contextual awareness of the project structure, ensuring that translated components integrate seamlessly with the overall repository architecture. The agent leverages its action space in an iterative manner, continuously reading source files to understand implementation details, creating translated files with language-appropriate adaptations, executing commands to validate translation correctness, and refining the implementation based on execution feedback until the translation task is completed or a timeout occurs.

V. EXPERIMENTAL SETUP

A. Research Questions

We aim to answer the following key research questions (RQs) that explore the utility of RepoTransBench and RepoTransAgent:

- **RQ1 (Performance of LLM-based Translation Methods):** How do different LLM-based translation methods perform in repository-level code translation tasks?
- **RQ2 (Performance Differences Across translation pairs):** What are the performance differences across different programming translation pairs?

- **RQ3 (Impact of Dependency Complexity):** How does dependency complexity affect the difficulty and performance of repository-level code translation?
- **RQ4 (Error Analysis):** What are the main types of errors that occur in repository-level code translation and what are their underlying causes?

B. Model Selection

As shown in Table III, we select 8 advanced LLMs from four different companies as our subject LLMs, which include 4 open-source and 4 closed-source LLMs representing the state-of-the-art in large language model capabilities.

For open-source LLMs, we include Alibaba's Qwen3-235B-A22B and its reasoning variant Qwen3-235B-A22B-think, both released in April 2025 with 235B parameters and 32K context windows. These models represent Alibaba's latest advancement in large-scale language modeling. Additionally, we evaluate DeepSeek's two flagship models: DeepSeek-Chat and DeepSeek-Reasoner, both featuring 236B parameters with extended 128K context windows. DeepSeek-Chat was released in March 2025, while DeepSeek-Reasoner, released in May 2025, incorporates enhanced reasoning capabilities that have demonstrated superior performance on complex reasoning tasks.

For closed-source LLMs, we examine four leading models from major AI companies. From Anthropic, we evaluate Claude-Sonnet-4, released in May 2025 with a 200K context window, representing one of the most capable reasoning models available. Google's Gemini-2.5-Flash-Lite, released in June 2025 with a 64K context window, offers efficient performance with reduced computational requirements. From OpenAI, we include two models: GPT-4.1, released in April 2025 with an impressive 1M context window enabling processing of extremely long documents, and o3-mini, released in January 2025 with a 200K context window, designed for efficient reasoning tasks.

Notably, most of the LLMs used in our experiments support substantial context windows ranging from 32K to 1M tokens, which significantly aids in understanding and processing long

TABLE III
THE SELECTION OF BACKBONE LLMs

Source	Model Name	Company	Size	Context Window	Release Date
Open Source	Qwen3-235B-A22B	Alibaba	235B	32K	Apr 2025
	Qwen3-235B-A22B-think	Alibaba	235B	32K	Apr 2025
	DeepSeek-Chat	DeepSeek	236B	128K	Mar 2025
	DeepSeek-Reasoner	DeepSeek	236B	128K	May 2025
Closed Source	Claude-Sonnet-4	Anthropic	-	200K	May 2025
	Gemini-2.5-Flash-Lite	Google	-	64K	Jun 2025
	GPT-4.1	OpenAI	-	1M	Apr 2025
	o3-mini	OpenAI	-	200K	Jan 2025

sequences of text or code. This extended context capability is particularly valuable for tasks requiring comprehensive understanding of large-scale content or complex multi-step reasoning.

C. Evaluation Metrics

We evaluate the translation and debugging performance by the following metrics:

- **SR** (Success Rate): The metric SR measures the percentage of translation tasks that successfully **pass all test cases**. A translation task is considered successful if and only if all test cases in the task are passed.

Let T_i represent the number of test cases that pass for the i -th translation task, and N_i represent the total number of test cases for the i -th translation task. The success indicator for the i -th task is defined as:

$$S_i = \begin{cases} 1 & \text{if } T_i = N_i \\ 0 & \text{otherwise} \end{cases} \quad (1)$$

The Success Rate is calculated as:

$$SR = \frac{1}{R} \sum_{i=1}^R S_i \quad (2)$$

where R is the total number of translation tasks (repositories) in the benchmark.

- **CR** (Compilation Rate): The metric CR measures the percentage of translation tasks that **successfully compile** without any compilation errors.

Let C_i represent the compilation indicator for the i -th translation task:

$$C_i = \begin{cases} 1 & \text{if the } i\text{-th task compiles successfully} \\ 0 & \text{otherwise} \end{cases} \quad (3)$$

The Compilation Rate is calculated as:

$$CR = \frac{1}{R} \sum_{i=1}^R C_i \quad (4)$$

where R is the total number of translation tasks in the benchmark.

- **APR** (Average Pass Rate): The metric APR measures the **average percentage of test cases passed** across all

translation tasks. It reflects the fine-grained performance at the individual test case level.

$$APR = \frac{1}{R} \sum_{i=1}^R \frac{T_i}{N_i} \quad (5)$$

where T_i represents the number of passed test cases for the i -th translation task, N_i represents the total number of test cases for the i -th translation task, and R is the total number of translation tasks.

- **AMPR** (Average Module Pass Rate): The metric AMPR measures the **average percentage of test modules passed** across all translation tasks. A test module is considered passed if and only if all test cases within that module are passed.

Let M_i represent the total number of test modules for the i -th translation task, and P_i represent the number of modules that pass all their test cases. The module pass indicator for the j -th module in the i -th task is defined as:

$$M_{i,j} = \begin{cases} 1 & \text{if all test cases in module } j \text{ of task } i \text{ pass} \\ 0 & \text{otherwise} \end{cases} \quad (6)$$

Then $P_i = \sum_{j=1}^{M_i} M_{i,j}$, and the Average Module Pass Rate is calculated as:

$$AMPR = \frac{1}{R} \sum_{i=1}^R \frac{P_i}{M_i} \quad (7)$$

where R is the total number of translation tasks in the benchmark.

It is worth noting that many previous works use **similarity-based metrics** like *BLEU* [90], *CodeBLEU* [8], or other metrics which calculate the overlapping tokens between references and translations [10], [12], [40], [91], [92] to evaluate the quality of translations. However, similarity-based metrics ignore the **syntactic correctness** and **functional correctness** of translations [34]. On the one hand, translations with high similarity to the reference cannot avoid having grammatical errors and functional errors. On the other hand, equivalent programs with different implementations may have low similarity. Therefore, we decide to use **execution-based metrics** mentioned above to evaluate the performance.

TABLE IV
PERFORMANCE EVALUATION RESULTS ACROSS DIFFERENT LLMs AND METHODS

Method	SR	CR	APR	AMPR	Method	SR	CR	APR	AMPR
TranslationOnly _{Qwen3}	0.0%	26.2%	18.6%	16.2%	TranslationOnly _{Claude}	0.0%	28.0%	16.4%	14.7%
ErrorFeedback _{Qwen3}	12.4%	30.5%	23.0%	20.5%	ErrorFeedback _{Claude}	11.3%	37.5%	26.8%	24.3%
RepoTransAgent _{Qwen3}	16.9%	34.4%	26.4%	23.6%	RepoTransAgent _{Claude}	32.8%	54.8%	44.8%	41.3%
TranslationOnly _{Qwen3-think}	0.0%	25.9%	18.7%	16.7%	TranslationOnly _{Gemini}	0.0%	32.5%	6.1%	5.2%
ErrorFeedback _{Qwen3-think}	13.8%	30.9%	22.9%	20.8%	ErrorFeedback _{Gemini}	4.1%	31.2%	10.3%	9.2%
RepoTransAgent _{Qwen3-think}	19.1%	36.0%	27.3%	25.0%	RepoTransAgent _{Gemini}	11.3%	34.4%	21.6%	19.9%
TranslationOnly _{DeepSeek}	0.0%	27.0%	17.2%	15.2%	TranslationOnly _{GPT-4.1}	0.0%	26.5%	19.6%	17.4%
ErrorFeedback _{DeepSeek}	13.9%	30.2%	24.3%	21.6%	ErrorFeedback _{GPT-4.1}	15.6%	37.7%	29.0%	26.1%
RepoTransAgent _{DeepSeek}	22.5%	36.5%	30.4%	27.9%	RepoTransAgent _{GPT-4.1}	32.8%	53.3%	45.4%	40.8%
TranslationOnly _{DeepSeek-R}	0.0%	10.9%	1.5%	1.3%	TranslationOnly _{o3-mini}	0.2%	33.3%	6.0%	5.4%
ErrorFeedback _{DeepSeek-R}	0.9%	10.8%	2.3%	1.9%	ErrorFeedback _{o3-mini}	8.7%	38.5%	11.4%	10.8%
RepoTransAgent _{DeepSeek-R}	1.2%	10.9%	2.5%	2.1%	RepoTransAgent _{o3-mini}	12.0%	41.0%	24.2%	23.0%

D. Execution Environment

To prevent LLMs from generating malicious code that could execute on the local machine and cause damage, we run the generated code in a sandbox (an isolated environment). We use *Docker* [93] as our code execution space, and bridge Docker's network with the local machine, allowing it to access the internet to ensure the dependencies in the "pom.xml" file can be successfully installed.

VI. EVALUATION RESULTS

A. RQ1: Performance of LLM-Based Translation Methods

Table IV presents comprehensive evaluation results across different LLMs and methodologies on our RepoTransBench benchmark. The results reveal significant variations in repository-level code translation capabilities and highlight the effectiveness of our proposed approach.

The results demonstrate that repository-level code translation remains a challenging task for current LLMs. Even the best-performing configuration (RepoTransAgent with Claude and GPT-4.1) achieves only 32.8% success rate, indicating that existing models struggle significantly with the complexity of entire software repositories. However, our RepoTransAgent framework consistently demonstrates superior performance across all evaluated models. For instance, with Claude, RepoTransAgent achieves 32.8% SR compared to 0.0% for TranslationOnly and 11.3% for ErrorFeedback, while with GPT-4.1, it achieves 32.8% SR versus 0.0% and 15.6% respectively. This consistent improvement across different LLMs validates the effectiveness of our agent-based approach for repository-level code translation.

Finding 1: Our RepoTransAgent framework consistently outperforms baseline approaches across all evaluated backbone models. However, repository-level code translation remains highly challenging for current LLM-based methods, with the best-performing method achieving only a 32.8% success rate.

The evaluation reveals a substantial performance gap between leading closed-source models and open-source alternatives. Claude and GPT-4.1 demonstrate superior performance with RepoTransAgent achieving 32.8% SR for both models, significantly outperforming open-source models such as Qwen3 (16.9%), DeepSeek-Chat (22.5%), and Gemini (11.3%). Notably, reasoning-focused models do not show clear advantages over their standard counterparts. Qwen3-think achieves 19.1% compared to Qwen3's 16.9%, representing only a modest improvement. More surprisingly, DeepSeek-Reasoner performs dramatically worse than DeepSeek-Chat (1.2% vs 22.5%), which can be attributed to its excessively long reasoning chains that lead to context management difficulties, request timeouts, and other technical issues that hinder effective translation performance.

Finding 2: Open-source models generally underperform leading closed-source models (Claude and GPT-4.1). Reasoning-based models exhibit no clear advantages, with DeepSeek-Reasoner performing significantly worse than DeepSeek-Chat due to context management and request timeout issues resulting from overly long reasoning chains.

An important observation is that direct translation approaches (TranslationOnly) consistently fail to produce repositories that pass all test cases, with 0.0% success rate across most models (except o3-mini with 0.2%). However, these approaches can still achieve partial success, with compilation rates ranging from 10.9% (DeepSeek-Reasoner) to 33.3% (o3-mini), and some test cases passing as evidenced by non-zero APR values. For example, TranslationOnly with Claude achieves 28.0% compilation rate and 16.4% average pass rate, indicating that while complete functional correctness is extremely difficult to achieve through direct translation, partial syntactic correctness and limited functional correctness are attainable.

TABLE V
PERFORMANCE OF REPOTRANSAGENT ACROSS DIFFERENT TRANSLATION PAIRS

Translation Pair	RepoTransAgent _{Qwen3}				RepoTransAgent _{Qwen3-think}				RepoTransAgent _{DeepSeek}				RepoTransAgent _{DeepSeek-R}			
	SR	CR	APR	AMPR	SR	CR	APR	AMPR	SR	CR	APR	AMPR	SR	CR	APR	AMPR
C#→Java	8.2	8.2	8.2	8.2	9.3	9.3	9.3	9.3	9.3	10.3	10.3	10.3	3.1	3.1	3.1	3.1
C++→Python	58.0	85.6	76.0	71.0	50.3	83.4	71.3	66.2	55.8	81.8	72.7	69.1	0.6	5.0	1.5	1.5
C→Python	43.4	91.0	63.1	56.3	33.6	84.4	62.0	52.6	43.4	80.3	62.3	56.1	1.6	5.7	1.6	1.6
C→Rust	6.6	10.7	9.7	8.2	12.3	14.8	13.7	13.1	23.8	23.8	23.8	23.8	0.0	0.0	0.0	0.0
JavaScript→Python	28.0	82.0	48.2	40.5	24.3	77.8	50.5	39.1	32.8	79.4	55.4	45.5	8.5	90.5	19.4	15.7
Java→C#	0.0	8.9	0.0	0.0	0.7	4.1	0.7	0.7	5.5	7.5	7.5	7.5	0.0	0.0	0.0	0.0
Java→Go	17.1	20.5	19.5	17.8	12.3	13.7	13.5	13.0	19.9	24.0	23.4	19.9	0.0	0.0	0.0	0.0
Java→Python	43.8	80.8	70.3	67.8	43.2	82.9	70.2	67.6	50.7	80.8	71.9	69.1	0.7	6.2	2.0	2.0
Matlab→Python	21.9	59.4	39.4	34.2	15.6	59.4	36.1	31.1	26.6	53.1	39.0	36.9	0.0	12.5	0.0	0.0
Python→C++	0.6	3.5	0.6	0.6	0.6	7.0	0.6	0.6	1.2	4.7	1.2	1.2	0.0	0.0	0.0	0.0
Python→Go	12.3	14.0	13.4	12.9	9.4	9.9	9.9	9.4	14.0	18.1	17.5	15.2	0.0	0.0	0.0	0.0
Python→Java	0.6	1.2	1.2	1.2	1.2	1.2	1.7	1.2	1.8	2.9	2.3	1.8	0.0	0.0	0.0	0.0
Python→Rust	5.3	5.8	5.8	5.8	4.7	4.7	4.7	4.7	8.8	8.8	8.8	8.8	0.0	0.0	0.0	0.0

Translation Pair	RepoTransAgent _{Claude}				RepoTransAgent _{Gemini}				RepoTransAgent _{GPT-4.1}				RepoTransAgent _{o3-mini}			
	SR	CR	APR	AMPR	SR	CR	APR	AMPR	SR	CR	APR	AMPR	SR	CR	APR	AMPR
C#→Java	28.9	34.0	33.0	33.0	8.2	10.3	11.1	10.3	20.6	22.7	25.3	22.7	3.1	4.1	4.1	4.1
C++→Python	63.0	97.8	83.2	80.4	21.0	84.5	56.4	51.9	55.2	96.1	78.3	75.4	3.9	99.4	55.1	51.5
C→Python	61.5	98.4	79.7	72.8	37.7	77.0	56.3	50.0	54.9	90.2	76.9	68.5	37.7	87.7	57.6	52.4
C→Rust	47.5	57.4	54.3	48.8	22.1	27.0	25.2	24.6	47.5	59.0	54.0	49.2	67.2	70.5	69.1	68.2
JavaScript→Python	53.4	96.3	74.8	67.4	3.2	95.8	29.3	24.4	43.4	86.2	68.5	60.0	34.9	94.7	47.7	43.9
Java→C#	19.2	30.8	24.0	24.0	0.0	1.4	0.7	0.7	21.2	23.3	25.5	25.2	0.7	1.4	0.7	0.7
Java→Go	36.3	43.8	43.0	36.3	14.4	16.4	16.4	16.4	33.6	47.3	45.7	34.2	4.1	4.1	4.1	4.1
Java→Python	45.9	98.6	79.8	77.0	38.4	86.3	64.1	62.7	58.2	86.3	79.8	77.8	3.4	97.3	56.3	54.7
Matlab→Python	48.4	95.3	67.8	65.4	14.1	35.9	26.0	22.8	34.4	71.9	59.8	57.7	4.7	96.9	20.3	19.7
Python→C++	3.5	26.3	6.2	4.1	0.0	1.2	0.7	0.0	9.9	38.0	11.1	10.5	0.6	0.6	0.6	0.6
Python→Go	18.7	31.6	29.3	23.4	1.2	1.2	1.2	1.2	19.9	33.3	31.4	21.1	1.2	1.2	1.2	1.2
Python→Java	5.8	8.8	8.2	8.2	0.0	0.0	0.0	0.0	7.0	7.0	9.0	7.0	1.8	1.8	1.8	1.8
Python→Rust	11.7	17.5	17.1	15.2	1.2	1.8	1.8	1.8	26.3	35.7	34.5	32.7	1.8	1.8	1.8	1.8

Finding 3: The translation-only method rarely produces repositories that pass all test cases. However, it can still achieve partial success with some repositories being compilable and passing individual test cases, highlighting the gap between syntactic and complete functional correctness.

B. RQ2: Performance Differences Across Translation Pairs

Table V presents detailed performance analysis of RepoTransAgent across 13 different translation pairs and multiple LLMs. The results reveal significant insights about the inherent difficulties and characteristics of different programming language translation combinations.

The most striking pattern in our results is the fundamental asymmetry between translating from statically-typed languages to dynamically-typed languages versus the reverse direction. Translations from static languages (C, C++, Java) to Python consistently achieve high success rates across most models: Claude achieves 61.5% for C→Python, 63.0% for C++→Python, and 45.9% for Java→Python, while GPT-4.1 reaches 54.9%, 55.2%, and 58.2% respectively. This success stems from the fact that static languages provide explicit type

information, memory management details, and structured interfaces that can be effectively simplified and adapted to Python's more flexible paradigm.

Conversely, Python-to-static-language translations face severe challenges across all evaluated models. Python→Java achieves only 0.0-7.0% success rates, Python→C++ reaches merely 0.0-9.9%, and Python→Rust performs between 1.2-26.3%. This dramatic performance gap reflects the fundamental challenge of inferring static type information, memory management strategies, and explicit interface definitions from Python's dynamic and implicit programming model. The translation process must essentially reverse-engineer the implicit contracts and assumptions present in dynamically-typed code.

Finding 4: Translation from statically-typed languages to dynamically-typed achieves substantially higher success rates (45-63%) compared to the reverse direction (typically below 10%), revealing a fundamental asymmetry in translation difficulty due to the challenges of inferring explicit type and interface information from dynamic code.

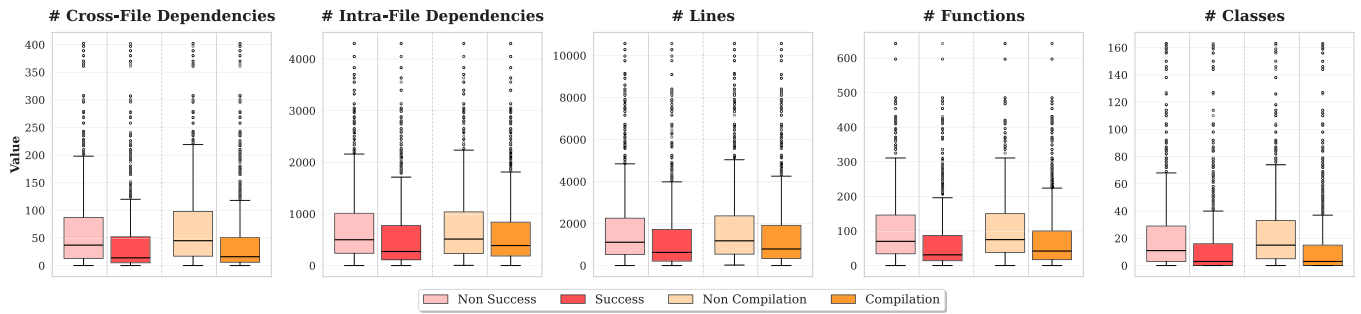


Fig. 4. The effect of code length and functional complexity on translation performance. The light colors (■ ■) represent non-successful outcomes, while dark colors (■ ■) represent successful outcomes. The first two groups (■ ■) in each subplot show success/non-success results, while the last two groups (■ ■) show compilation/non-compilation results. To improve clarity, outliers where metrics exceeded the 95th percentile have been omitted.

Our analysis reveals that different LLM backbones exhibit surprising specialization patterns for specific translation pairs, likely reflecting their training data composition and architectural biases. Most notably, o3-mini demonstrates exceptional performance on C→Rust translation with 67.2% success rate, significantly outperforming leading models like Claude (47.5%) and GPT-4.1 (47.5%) on the same pair. This suggests that o3-mini’s training included substantial Rust-related content or system-level programming examples that enhanced its understanding of memory safety patterns and ownership concepts critical for C-to-Rust translation.

Similarly, we observe that certain models excel at specific paradigm shifts while struggling with others. For instance, Claude shows strong performance across most translation pairs but particularly excels at translating to Python (achieving the highest success rates for C→Python, C++→Python, and Matlab→Python), suggesting robust training on Python codebases. In contrast, Gemini demonstrates highly inconsistent performance, achieving strong compilation rates for some pairs (95.8% for JavaScript→Python compilation) but near-zero functional success rates, indicating potential gaps in understanding semantic equivalence across languages. These patterns suggest that training data distribution and architectural choices significantly influence model performance on specific language combinations.

Finding 5: Different LLM backbones demonstrate specialized advantages for specific translation pairs (e.g., o3-mini excelling at C→Rust with 67.2% vs. Claude’s 47.5%), likely reflecting training data composition and architectural biases that favor particular programming languages.

C. RQ3: Impact of Dependency Complexity

Fig. 4 presents a comprehensive analysis of how repository complexity characteristics affect translation performance across different dimensions. We examine the relationship between various complexity metrics and translation outcomes to understand the fundamental challenges posed by repository-level code translation.

The analysis reveals a clear inverse relationship between repository complexity and translation success across multiple

dimensions. For cross-file dependencies, successful translations consistently exhibit lower median values compared to failed translations, indicating that repositories with fewer inter-module dependencies are more amenable to successful translation. Similarly, intra-file dependencies show that successful translations tend to have simpler internal dependency structures. The pattern extends to basic size metrics, where successful translations typically involve repositories with fewer lines of code, fewer functions, and fewer classes. This consistent trend across all complexity dimensions suggests that current LLMs struggle systematically with increased repository complexity, regardless of whether the complexity stems from structural dependencies, code volume, or functional richness.

Finding 6: Repository complexity across all dimensions (cross-file dependencies, intra-file dependencies, code length, and structural complexity) inversely correlates with translation success, indicating that current LLMs struggle systematically with complex repository structures.

D. RQ4: Error Analysis

We conduct an error analysis across three rounds of experiments, ultimately identifying common errors in repository-level code translation. These errors are classified into five categories: E1 (Configuration File Issues), E2 (Limited Understanding Ability Issues), E3 (Incomplete Generation Issues), E4 (Language Feature Issues), E5 (Encoding Issues). Due to space constraints, we provide two cases for each category.

E1. Configuration File Issues often arise when build-related content (e.g., “CMakeLists.txt” in a C++ CMake project) is not configured correctly. Fig. 5 shows an error due to an unresolved dependency as the package “nonexistent-lib” cannot be found through CMake’s package configuration system. Beyond this common error, we observe other configuration-related issues including version compatibility problems, missing dependency declarations in build files (e.g., “package.json”, “requirements.txt”, “Cargo.toml”), and platform-specific configuration errors. Some LLMs occasionally generate inappropriate content for configuration files, leading to build failures.


```

find_package(nonexistent-lib CONFIG REQUIRED)
find_package(outdated-boost 1.999.0 REQUIRED)

target_link_libraries(main PRIVATE nonexistent-
lib::nonexistent-lib)

[ERROR] CMake Error at CMakeLists.txt:10 (find_package): Could not find a
package configuration file provided by "nonexistent-lib"

```

Fig. 5. Error Type 1: Configuration File Issues.

```

type UrlEncoder struct {
    alphabet string
}
func (u *UrlEncoder) EncodeUrl(n int, minLength int)
string {
    return u.processUrl(n, minLength)
}
func (u *UrlEncoder) EncodeUrl(n int, minLength int)
string {
    return u.generateShortUrl(n, minLength)
}

[ERROR] ./main.go:12:1: UrlEncoder.EncodeUrl redeclared in this block
previous declaration at ./main.go:7:1

```

Fig. 6. Error Type 2: Limited Understanding Ability Issues.

Finding 7: Unlike fine-grained code translation works, repository-level code translation requires proper configuration of files such as the “CMakeLists.txt” file in C++ repositories. This can lead to dependency-related issues (e.g., non-existent dependencies, version mismatches, etc.). Solving these problems requires a clear understanding of the related calls and the latest dependencies.

E2. Limited Understanding Ability Issues usually arise due to unfamiliarity with the code context during translation. Fig. 6 shows two methods with identical names and parameter signatures in a Go struct, leading to a redeclaration error. This occurs because the previously generated method is overlooked when generating new functions, often due to insufficient context awareness in long codebases. Beyond this example, we observe other context-related issues including incorrect function calls with mismatched argument types, improper imports due to misunderstanding repository structure, and variable scope conflicts. These errors typically stem from LLMs’ limited ability to maintain comprehensive understanding of the entire codebase during translation.

Finding 8: Due to the long code length and complex repository structure, a lack of understanding of the repository context may result in generating inappropriate code. To mitigate these issues, taking measures to enhance the focus on relevant context within the repository might be helpful.

E3. Incomplete Generation Issues often occur due to LLMs’ limitation in instruction following and code generation abilities. Fig. 7 shows an example where the LLM fails to complete a long Rust function, leaving unclosed delimiters and incomplete statements. The generation terminates abruptly in the middle of a HashMap insertion, resulting in syntax errors due to missing closing braces and parentheses. Beyond incomplete

```

impl PhysicsConstants {
    pub fn set_derived_units_and_constants() {
        let m_s = 1e-3 * S;
        let u_s = 1e-6 * S;
        // ... (omitted 150+ lines of code)

        let conversion_table = HashMap::new();
        for unit in unit_list {
            conversion_table.insert(unit.name, unit.factor

[ERROR] error: this file contains an unclosed delimiter
--> src/physics_constants.rs:157:48

2 |     pub fn set_derived_units_and_constants() {
  |                                             - unclosed

```

Fig. 7. Error Type 3: Incomplete Generation Issues.

```

private class UrlEncoder { ... }

public static String encodeUrl(int n, int min_length) {
    UrlEncoder encoder =
        new UrlEncoder(DEFAULT_ALPHABET, DEFAULT_BLOCK_SIZE);
    ... }

[ERROR] src/main/java/ShortUrl.java:[36,30] non-static variable this cannot be
referenced from a static context

```

Fig. 8. Error Type 4: Language Feature Issues.

function bodies, we observe other generation issues including missing import statements for used packages, incomplete class or struct definitions, and truncated method implementations. These problems often occur when LLMs struggle with long code sequences or when they generate example-style code while omitting essential elements.

Finding 9: Some LLMs may struggle to continue generating due to limited instruction following capability, and others may tend to generate example code with some elements omitted. These issues can be automatically identified by dependency analysis and syntax checking. Then we can fix these issues by regenerating or replacing them with stronger LLMs.

E4. Language Feature Issues occur frequently in repository-level code translation due to its functional complexity. Fig. 8 shows a case where a static method attempts to access a non-static inner class `UrlEncoder`, which violates Java’s static context rules. This error demonstrates the fundamental misunderstanding of static versus non-static member accessibility in Java. Beyond this example, we observe other language-specific issues including attempts to instantiate abstract classes, direct access to private member variables from external classes, and incorrect use of language-specific keywords or modifiers. These problems typically arise when LLMs perform token-by-token translation without considering the target language’s semantic constraints and access control mechanisms.

Finding 10: When translating a repository to another language, some LLMs may lack sufficient understanding of language features, leading to related issues. Providing LLMs with more information about the language features in the prompt may help improve the translation performance.



Fig. 9. Error Type 5: Encoding Issues.

E5. Encoding Issues occur due to incompatible repository encoding formats and can lead to compilation failures when using special characters. Fig. 9 shows an example where a Java source file contains an emoji character in a string literal, but the compiler is configured to use US-ASCII encoding, which cannot handle Unicode characters beyond the basic ASCII range. Beyond emoji usage, we observe other encoding-related issues including problems with non-English characters in comments or string literals, issues when reading files with different encoding formats, and compilation errors in multilingual environments. These problems typically arise when the build system's default encoding configuration is insufficient for the characters present in the translated code.

Finding 11: Non-US-ASCII characters commonly occur in repositories, and encoding-related issues can sometimes arise when reading or writing resources. These problems can be resolved by configuring correct encoding format of the repository.

VII. THREATS TO VALIDITY

Internal Threats. The first potential internal threat concerns the *scope of evaluated LLMs*. While we evaluate 8 state-of-the-art LLMs across different categories (open-source, closed-source, and reasoning-focused models), the rapidly evolving landscape of LLMs means that newer models may exhibit different performance characteristics. However, our evaluation includes leading models from major providers (OpenAI, Anthropic, Google, Alibaba, DeepSeek) that represent the current state-of-the-art, ensuring broad coverage of existing capabilities. Additionally, we do not apply fine-tuning methods to these LLMs specifically for repository-level translation, which may impact their performance. This limitation is partially mitigated by including specialized code models that have been pre-trained on code-related tasks. Another potential threat is the *selection of programming translation pairs*. Our benchmark focuses on 13 translation pairs across 7 programming languages, with emphasis on commonly used languages in software development. While this covers major programming paradigms (object-oriented, functional, systems programming), the results may not generalize to less common languages or domain-specific languages. However, our language selection is based on TIOBE rankings and developer survey data, ensuring relevance to real-world translation needs. The inclusion of diverse language combinations (e.g., C ↔ Python, Java ↔ Go, Python ↔ Rust) provides insights into various translation scenarios encountered in practice.

External Threats. The primary external threat involves *LLMs' generation variability*. Large language models exhibit

inherent randomness in their outputs, which could affect the reproducibility of our results. To mitigate this threat, we employ consistent experimental settings and analyze performance patterns across multiple samples rather than isolated instances. Additionally, our evaluation focuses on objective metrics (compilation success, test passage) that are less susceptible to generation variance compared to subjective quality assessments. Another potential threat concerns the *comprehensiveness of functional correctness evaluation*. Our evaluation primarily relies on execution-based metrics using existing test suites from source repositories. While passing all test cases provides strong evidence of functional correctness, it may not capture all edge cases or guarantee complete semantic equivalence. However, this approach represents a significant advancement over similarity-based metrics used in previous work, as it directly validates the operational correctness of translated code. Real-world test suites from production repositories provide more realistic evaluation scenarios compared to artificially constructed benchmarks. The *repository selection and filtering process* may introduce bias toward certain types of projects. Our filtering criteria (star count, language composition, executability) may favor well-maintained, popular repositories while excluding experimental or domain-specific projects. This bias is intentional to ensure benchmark quality and practical relevance, as successful repository-level translation tools should prioritize handling well-structured, maintainable codebases that represent common development scenarios. Finally, the *temporal validity* of our benchmark presents a consideration, as programming languages, frameworks, and development practices evolve continuously. However, our focus on fundamental language features and well-established frameworks ensures that our findings remain relevant across reasonable time horizons. The automated benchmark construction framework we develop can facilitate future updates and extensions to maintain benchmark currency.

VIII. RELATED WORK

Many benchmarks have been introduced to compare the performance of different translation techniques objectively. CoST [32] and XLCost [33] introduce a snippet and function-level code translation benchmark. CodeXGLUE [35] includes a dataset for function-level Java-C# code translation. TransCoder-test is the evaluation dataset for TransCoder [34], which includes the function-level code translation on Python, Java, and C++. Some other benchmarks like HumanEval-X [36] source from HumanEval [56] to construct a function-level code translation benchmark. G-TransEval [67] provides a more fine-grained taxonomy, including token-level, syntax-level, library-level, and algorithm-level, which is part of a function. CodeNet [37], Avatar [38], xCodeEval [58] CodeScope [39] and CodeTransOcean [40] introduce file-level code translation benchmarks which source from code contest platforms like codeforces [60], atcoder [61], aizuc [62], Google Code Jam [63], etc. or task solutions websites like samples from.Net [64], d2l.ai [65], rosetta code [66], etc. Although these benchmarks can evaluate the capabilities of existing code translation techniques to some extent, they cannot evaluate the performance of current

techniques on real-world repository-level code translation tasks. Recently, Pan et al. [1] manually study two open-source repositories (Apache Commons CLI [41] and Python Click [42]) and find that current LLMs struggle to complete the translation tasks of entire repositories. However, they do not provide a sufficient number of repositories and corresponding automatic test suites for evaluation. Besides, the resource and configuration files are ignored in this research. Recently, Pan et al. [1] manually study two open-source repositories (Apache Commons CLI [41] and Python Click [42]) and find that current LLMs struggle to complete the translation tasks of entire repositories. AlphaTrans [94] is proposed as a neuro-symbolic compositional technique that decomposes repositories into fragments and translates them in reverse call order with GraalVM-based validation. TRACY [95] is a benchmark proposed to evaluate the execution efficiency of function-level code translation. However, our main contribution is introducing a real-world multilingual benchmark for verifying the equivalence of repository-level code translation.

IX. CONCLUSION

This paper addresses the gap between existing fine-grained code translation benchmarks and real-world software development demands by introducing RepoTransBench, a comprehensive repository-level benchmark with 1,897 samples across 13 translation pairs, and RepoTransAgent, an intelligent agent framework based on the ReAct paradigm for systematic repository translation. Our evaluation reveals that repository-level translation remains challenging, with the best-performing method achieving only a 32.8% success rate. We also observe strong directional asymmetry in translation difficulty (static-to-dynamic achieving 45-63% vs. reverse direction below 10%), model-specific advantages for certain translation pairs reflecting training biases, and inverse correlation between repository complexity and translation success. This paper provides the community with both a challenging benchmark and practical guidance for future research.

REFERENCES

- [1] R. Pan et al., "Lost in translation: A study of bugs introduced by large language models while translating code," in *Proc. IEEE/ACM 46th Int. Conf. Softw. Eng.*, 2024, pp. 1–13.
- [2] Q. Sun et al., "A survey of neural code intelligence: Paradigms, advances and beyond," 2024, *arXiv:2403.14734*.
- [3] H. F. Eniser et al., "Towards translating real-world code with LLMs: A study of translating to rust," 2024, *arXiv:2405.11514*.
- [4] S. Dou et al., "What's wrong with your code generated by large language models? An extensive study," 2024, *arXiv:2407.06153*.
- [5] J. Austin et al., "Program synthesis with large language models," 2021, *arXiv:2108.07732*.
- [6] E. Nijkamp et al., "CodeGen: An open large language model for code with multi-turn program synthesis," 2022, *arXiv:2203.13474*.
- [7] Y. Xie, A. Naik, D. Fried, and C. Rose, "Data augmentation for code translation with comparable corpora and multiple references," 2023, *arXiv:2311.00317*.
- [8] S. Ren et al., "CodeBLEU: A method for automatic evaluation of code synthesis," 2020, *arXiv:2009.10297*.
- [9] D. Hendrycks et al., "Measuring coding challenge competence with apps," 2021. [Online]. Available: <https://arxiv.org/abs/2105.09938>
- [10] K. Aggarwal, M. Salameh, and A. Hindle, "Using machine translation for converting python 2 to python 3 code," *PeerJ PrePrintsTechRep.*, 2015.
- [11] M. Szafraniec, B. Roziere, H. Leather, F. Charton, P. Labatut, and G. Synnaeve, "Code translation with compiler representations," 2022, *arXiv:2207.03578*.
- [12] A. T. Nguyen, T. T. Nguyen, and T. N. Nguyen, "Lexical statistical machine translation for language migration," in *Proc. 9th Joint Meeting Foundations Softw. Eng.*, 2013, pp. 651–654.
- [13] A. T. Nguyen, T. T. Nguyen, and T. N. Nguyen, "Migrating code with statistical machine translation," in *Proc. 36th Int. Conf. Softw. Eng. Companion*, 2014, pp. 544–547.
- [14] M. Mossienko, "Automated Cobol to java recycling," in *Proc. 7th Eur. Conf. Softw. Maintenance Reeng.*, Piscataway, NJ, USA: IEEE Press, 2003, pp. 40–50.
- [15] A. E. Hassan and R. C. Holt, "A lightweight approach for migrating web frameworks," *Inf. Softw. Technol.*, vol. 47, no. 8, pp. 521–532, 2005.
- [16] T. T. Bartolomei, K. Czarnecki, and R. Lämmel, "Swing to SWT and back: Patterns for API migration by wrapping," in *Proc. IEEE Int. Conf. Softw. Maintenance*, Piscataway, NJ, USA: IEEE Press, 2010, pp. 1–10.
- [17] H. Zhong, S. Thummalapenta, T. Xie, L. Zhang, and Q. Wang, "Mining API mapping for language migration," in *Proc. 32nd ACM/IEEE Int. Conf. Softw. Eng.*, 2010, vol. 1, pp. 195–204.
- [18] A. T. Nguyen, H. A. Nguyen, T. T. Nguyen, and T. N. Nguyen, "Statistical learning approach for mining API usage mappings for code migration," in *Proc. 29th ACM/IEEE Int. Conf. Automated Softw. Eng.*, 2014, pp. 457–468.
- [19] X. Gu, H. Zhang, D. Zhang, and S. Kim, "DeepAM: Migrate APIS with multi-modal sequence to sequence learning," 2017, *arXiv:1704.07734*.
- [20] D. Bahdanau, "Neural machine translation by jointly learning to align and translate," 2014, *arXiv:1409.0473*.
- [21] X. Chen, C. Liu, and D. Song, "Tree-to-tree neural networks for program translation," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 31, 2018, pp. 2552–2562.
- [22] M. Artetxe, G. Labaka, and E. Agirre, "Unsupervised statistical machine translation," 2018, *arXiv:1809.01272*.
- [23] J. Devlin, "Bert: Pre-training of deep bidirectional transformers for language understanding," 2018, *arXiv:1810.04805*.
- [24] Z. Feng et al., "CodeBert: A pre-trained model for programming and natural languages," 2020, *arXiv:2002.08155*.
- [25] D. Guo et al., "GraphCodeBert: Pre-training code representations with data flow," 2020, *arXiv:2009.08366*.
- [26] W. U. Ahmad, S. Chakraborty, B. Ray, and K.-W. Chang, "Unified pre-training for program understanding and generation," 2021, *arXiv:2103.06333*.
- [27] D. Guo, S. Lu, N. Duan, Y. Wang, M. Zhou, and J. Yin, "Unix-coder: Unified cross-modal pre-training for code representation," 2022, *arXiv:2203.03850*.
- [28] Q. Zheng et al., "CodeGEE: A pre-trained model for code generation with multilingual benchmarking on humaneval-x," in *Proc. 29th ACM SIGKDD Conf. Knowl. Discovery Data Mining*, 2023, pp. 5673–5684.
- [29] Z. Yang et al., "Exploring and unleashing the power of large language models in automated code translation," *Proc. ACM Softw. Eng.*, vol. 1, no. FSE, pp. 1585–1608, 2024.
- [30] K. Lano and H. Siala, "Using model-driven engineering to automate software language translation," *Automated Softw. Eng.*, vol. 31, no. 1, p. 20, 2024.
- [31] V. Nitin and B. Ray, "Spectra: Enhancing the code translation ability of language models by generating multi-modal specifications," 2024, *arXiv:2405.18574*.
- [32] M.-Y. Zhu, K. Suresh, and C. K. Reddy, "Multilingual code snippets training for program translation," in *Proc. AAAI Conf. Artif. Intell.*, 2022. [Online]. Available: <https://api.semanticscholar.org/CorpusID:245216916>
- [33] M. Zhu, A. Jain, K. Suresh, R. Ravindran, S. Tipirneni, and C. K. Reddy, "Xlcost: A benchmark dataset for cross-lingual code intelligence," 2022, *arXiv:2206.08474*.
- [34] M.-A. Lachaux, B. Roziere, L. Chassusot, and G. Lample, "Unsupervised translation of programming languages," 2020, *arXiv:2006.03511*.
- [35] S. Lu et al., "CodeXGLUE: A machine learning benchmark dataset for code understanding and generation," 2021, *arXiv:2102.04664*.
- [36] "humaneval-x." Accessed: Nov. 10, 2025. [Online]. Available: <https://huggingface.co/datasets/THUDM/humaneval-x>
- [37] R. Puri et al., "CodeNet: A large-scale AI for code dataset for learning a diversity of coding tasks," 2021, *arXiv:2105.12655*.
- [38] W. U. Ahmad, M. G. R. Tushar, S. Chakraborty, and K.-W. Chang, "Avatar: A parallel corpus for java-python program translation," 2021, *arXiv:2108.11590*.

- [39] W. Yan et al., "Codescope: An execution-based multilingual multitask multidimensional benchmark for evaluating LLMs on code understanding and generation," 2023, *arXiv:2311.08588*.
- [40] W. Yan, Y. Tian, Y. Li, Q. Chen, and W. Wang, "Codetransocean: A comprehensive multilingual benchmark for code translation," 2023, *arXiv:2310.04951*.
- [41] "Apache commons cli." Accessed: Nov. 10, 2025. [Online]. Available: <https://commons.apache.org/proper/commons-cli/>
- [42] "Click." Accessed: Nov. 10, 2025. [Online]. Available: <https://click.palletsprojects.com/en/8.1.x/>
- [43] "Antlr." Accessed: Nov. 10, 2025. [Online]. Available: <https://wwwantlr.org/>
- [44] "Babel." Accessed: Nov. 10, 2025. [Online]. Available: <https://babeljs.io/>
- [45] "Emscripten." Accessed: Nov. 10, 2025. [Online]. Available: <https://emscripten.org/>
- [46] "Jsweet." Accessed: Nov. 10, 2025. [Online]. Available: <https://www.jsweet.org/>
- [47] "Gwt." Accessed: Nov. 10, 2025. [Online]. Available: <https://www.gwtproject.org/overview.html>
- [48] "Cxgo." Accessed: Nov. 10, 2025. [Online]. Available: <https://github.com/gotranspile/cxgo>
- [49] "c2rust." Accessed: Nov. 10, 2025. [Online]. Available: <https://github.com/immunant/c2rust>
- [50] "Javatocsharp." Accessed: Nov. 10, 2025. [Online]. Available: <https://github.com/paulirwin/JavaToCSharp>
- [51] "Haxe." Accessed: Nov. 10, 2025. [Online]. Available: <https://haxe.org/>
- [52] "Swig." Accessed: Nov. 10, 2025. [Online]. Available: <https://swig.org/>
- [53] Y. Wang, W. Wang, S. Joty, and S. C. Hoi, "Codet5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation," 2021, *arXiv:2109.00859*.
- [54] P. Koehn et al., "Moses: Open source toolkit for statistical machine translation," in *Proc. 45th Annu. Meeting Assoc. Comput. Linguistics Companion Vol. Proc. Demo Poster Sessions*, 2007, pp. 177–180.
- [55] H. Zheng, Y. Cheng, and Y. Liu, "Maximum expected likelihood estimation for zero-resource neural machine translation," in *Proc. IJCAI*, 2017, pp. 4251–4257.
- [56] M. Chen et al., "Evaluating large language models trained on code," 2021, *arXiv:2107.03374*.
- [57] "Geeksforgereeks," [Online]. Available: <https://www.geeksforgereeks.org/>
- [58] M. A. M. Khan, M. S. Bari, X. L. Do, W. Wang, M. R. Parvez, and S. Joty, "xcodetval: A large scale multilingual multitask benchmark for code understanding, generation, translation and retrieval," 2023, *arXiv:2303.03004*.
- [59] X. Yin, C. Ni, T. N. Nguyen, S. Wang, and X. Yang, "Rectifier: Code translation with corrector via LLMs," 2024, *arXiv:2407.07472*.
- [60] "Codeforces." Accessed: Nov. 10, 2025. [Online]. Available: <https://codeforces.com/>
- [61] "Atcoder." Accessed: Nov. 10, 2025. [Online]. Available: <https://atcoder.jp/>
- [62] "Aizu." Accessed: Nov. 10, 2025. [Online]. Available: <https://onlinejudge.u-aizu.ac.jp>
- [63] "Googlecodejam." Accessed: Nov. 10, 2025. [Online]. Available: <https://codingcompetitionsonair.withgoogle.com/>
- [64] "Dotnetsamples." Accessed: Nov. 10, 2025. [Online]. Available: <https://learn.microsoft.com/en-us/samples/dotnet/try-samples/101-linqsamples/>
- [65] "d2l.ai." Accessed: Nov. 10, 2025. [Online]. Available: <https://github.com/d2l-ai/d2l-zh>
- [66] "Rosettacode." Accessed: Nov. 10, 2025. [Online]. Available: https://rosettacode.org/wiki/Rosetta_Code
- [67] M. Jiao, T. Yu, X. Li, G. Qiu, X. Gu, and B. Shen, "On the evaluation of neural code translation: Taxonomy and benchmark," in *Proc. 38th IEEE/ACM Int. Conf. Automated Softw. Eng. (ASE)*, Piscataway, NJ, USA: IEEE Press, 2023, pp. 1529–1541.
- [68] R. Li et al., "Starcode: May the source be with you!," 2023, *arXiv:2305.06161*.
- [69] A. Lozhkov et al., "Starcode 2 and the stack v2: The next generation," 2024, *arXiv:2402.19173*.
- [70] L. B. Allal et al., "Santacoder: Don't reach for the stars!," 2023, *arXiv:2301.03988*.
- [71] H. Touvron et al., "Llama: Open and efficient foundation language models," 2023, *arXiv:2302.13971*.
- [72] H. Touvron et al., "Llama 2: Open foundation and fine-tuned chat models," 2023, *arXiv:2307.09288*.
- [73] A. Dubey et al., "The llama 3 herd of models," 2024, *arXiv:2407.21783*.
- [74] OpenAI, "Gpt-4 technical report," 2023. Available: <https://arxiv.org/abs/2303.08774>.
- [75] DeepSeek-AI, "Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model," 2024.
- [76] Anthropic, "The Claude 3 model family: Opus, sonnet, haiku." [Online]. Available: <https://api.semanticscholar.org/CorpusID:268232499>
- [77] A. Z. Yang, Y. Takashima, B. Paulsen, J. Dodds, and D. Kroening, "Vert: Verified equivalent rust transpilation with few-shot learning," 2024, *arXiv:2404.18852*.
- [78] M. Bhattarai, J. E. Santos, S. Jones, A. Biswas, B. Alexandrov, and D. O'Malley, "Enhancing code translation in language models with few-shot learning via retrieval-augmented generation," 2024, *arXiv:2407.19619*.
- [79] Z. Yuan, W. Chen, H. Wang, K. Yu, X. Peng, and Y. Lou, "TransAGENT: An LLM-based multi-agent system for code translation," 2024, *arXiv:2409.19894*.
- [80] M. Shiraishi and T. Shinagawa, "Context-aware code segmentation for c-to-rust translation using large language models," 2024, *arXiv:2409.10506*.
- [81] R. Karanjai, L. Xu, and W. Shi, "Teaching machines to code: Smart contract translation with LLMs," 2024, *arXiv:2403.09740*.
- [82] "Solidity." Accessed: Nov. 10, 2025. [Online]. Available: <https://soliditylang.org/>
- [83] "Move." Accessed: Nov. 10, 2025. [Online]. Available: <https://github.com/move-language/move>
- [84] W. Yang, Y. Guo, and Y. Xue, "CCTrans: A java-to-javascript translation with concurrency runtime," 2024, *arXiv:2408.09481*.
- [85] M. S. Abid et al., "Glutest: Testing code translation via language interoperability," in *Proc. IEEE Int. Conf. Softw. Maintenance Evol. (ICSME)*, 2024, pp. 612–617, doi: 10.1109/ICSME58944.2024.00061.
- [86] F. Liu, J. Li, and L. Zhang, "Syntax and domain aware model for unsupervised program translation," in *Proc. IEEE/ACM 45th Int. Conf. Softw. Eng. (ICSE)*, Piscataway, NJ, USA: IEEE Press, 2023, pp. 755–767.
- [87] "maven." Accessed: Nov. 10, 2025. [Online]. Available: <https://maven.apache.org/>
- [88] "Tiobe-index." Accessed: Nov. 10, 2025. [Online]. Available: <https://www.tiobe.com/tiobe-index/>
- [89] "GitHub." Accessed: Nov. 10, 2025. [Online]. Available: <https://github.com/DeepSoftwareAnalytics/RepoTransBench>
- [90] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, "Bleu: a method for automatic evaluation of machine translation," in *Proc. 40th Annu. Meeting Assoc. Comput. Linguistics*, 2002, pp. 311–318.
- [91] S. Karaivanov, V. Raychev, and M. Vechev, "Phrase-based statistical translation of programming languages," in *Proc. ACM Int. Symp. New Ideas, New Paradigms, Reflections Program. Softw.*, 2014, pp. 173–184.
- [92] A. V. M. Barone and R. Sennrich, "A parallel corpus of python functions and documentation strings for automated code documentation and code generation," 2017, *arXiv:1707.02275*.
- [93] "Docker." Accessed: Nov. 10, 2025. [Online]. Available: <https://www.docker.com/>
- [94] A. R. Ibrahimzade et al., "Alphatrans: A neuro-symbolic compositional approach for repository-level code translation and validation," *Proc. ACM Softw. Eng.*, vol. 2, no. FSE, pp. 2454–2476, 2025.
- [95] Z. Gong, Z. Sun, D. Huang, Q. Liang, J. M. Zhang, and D. Hao, "Tracy: Benchmarking execution efficiency of LLM-based code translation," 2025, *arXiv:2508.11468*.